ADDIS ABABA UNIVERISITY

COLLEGE OF TECHNOLOGY AND BUILT ENVIRONMENT

SCHOOL OF INFORMATION TECHNOLOGY AND ENGINEERING

Project Proposal

AI ENHANCED MODSECURITY

PREPARED BY:

1. Eyoab Amare UGR/5756/14
2. Meaza Tadele UGR/6378/14
3. Fozia Mohammed UGR/4535/14
4. Natanim Kemal UGR/4648/14
5. Liyu Desta UGR/6694/13

ACKNOWLEDGMENT

We would like to express our sincere gratitude to our advisor for his continuous support and feedback throughout the development of this proposal. His guidance helped shape this project idea and made it possible to clearly identify the need for an improved, intelligent intrusion detection solution.

TABLE OF CONTENTS

ACKNOWLEDGMENT 2
ACRONYMS 4
ABSTRACT 5
1. INTRODUCTION 6
1.1 Background 6
1.2 The Existing System 6
1.3 Statement of the Problem 7
1.4 Objective of the Project 8
1.4.1 General Objective 8
1.4.2 Specific Objectives 8
1.5 Proposed System 8
1.6 Feasibility Study 9
1.6.1 Economic Feasibility 9
1.6.2 Technical Feasibility 9
1.6.3 Schedule Feasibility 9
1.7 Scope 9
1.8 Methodology 10
1.9 Project Management Plan 14
1.9.1 Time Management Plan (Gantt Overview) 14
1.9.2 Quality Management Plan 14
1.9.3 Communication Management Plan 15
REFERENCE 15

ACRONYMS

Acronym	Meaning
WAF	Web Application Firewall
IDS	Intrusion Detection System
CRS	Core Rule Set
FP	False Positive
TP	True Positive
AI	Artificial Intelligence
ML	Machine Learning
HTTP	Hypertext Transfer Protocol

ABSTRACT

ModSecurity is one of the most widely used open-source Web Application Firewalls, applying rule-based detection mechanisms to protect web applications from attacks such as SQL Injection, Cross-Site Scripting, and malicious payloads. However, one of the major operational challenges of ModSecurity is the high rate of false positives, which often leads to legitimate traffic being flagged as malicious. This results in user inconvenience, system downtime, and excessive work for security administrators.

This project proposes an AI-enhanced ModSecurity system that integrates machine learning to reduce false positives without compromising security. The system will collect ModSecurity audit logs, train an AI model to classify alerts into true positives and false positives, and act as an intelligent triage layer to reduce noise. The project will demonstrate how ML can augment traditional rule-based IDS systems and provide measurable improvements in accuracy, operational efficiency, and alert quality.

1. INTRODUCTION

1.1 Background

Web applications are increasingly exposed to cyberattacks, making Web Application Firewalls (WAFs) essential components of modern security infrastructures. ModSecurity, combined with the OWASP Core Rule Set (CRS), provides a robust rule-based mechanism to detect and block known web attacks. However, as applications become more complex and traffic behaviors evolve, static rules often misclassify legitimate requests as malicious.

Organizations, ranging from small businesses to large enterprises, face the burden of managing large volumes of false alerts. This reduces trust in the system and increases security overhead. Integrating AI into WAFs is a modern approach that helps systems adapt to evolving traffic patterns.

This project will deliver an AI-powered extension to ModSecurity designed to automatically analyze, classify, and reduce false positives, thereby improving accuracy and making WAF management more efficient.

1.2 The Existing System

Currently, ModSecurity operates purely through **static rules** found in the OWASP CRS. These rules detect attacks using predefined patterns and regular expressions.

Existing workflow:

- ModSecurity inspects HTTP requests.
- If a rule matches, an alert is triggered.
- Administrators manually review logs.
- Exclusion rules are added to avoid repeated false positives.

Problems in the existing process:

- High number of False positive
- Manual tuning is time-consuming and error prone.
- Rule based systems cannot adapt to new or application specific traffic.
- Lack of intelligent triage leads to alert fatigue.

This gap motivates the development of an AI-assisted solution.

1.3 Statement of the Problem

The widespread deployment of Web Application Firewalls based on ModSecurity and the OWASP Core Rule Set (CRS) has revealed a persistent and critical limitation: an excessively high rate of false positive alerts. This problem has existed for over 15 years, since ModSecurity was first released in 2005 and the CRS was introduced in 2006, and continues to worsen with the increasing complexity of modern web applications (dynamic content, APIs, JSON payloads, single-page applications).

Element	Description
The problem of ...	ModSecurity frequently generates a high volume of false positive alerts, especially in applications with dynamic or JSON-heavy traffic.
Affects ...	• Web administrators • Security analysts • End-users blocked during legitimate actions • Organizations relying on ModSecurity for protection
And results in ...	• Alert fatigue and increased workload • Legitimate traffic being denied • Reduced confidence in the WAF • Delays in identifying real threats due to noise
Benefits of a solution ...	• Automatic reduction of false positives • Improved ModSecurity accuracy • Less manual intervention • Faster threat detection and response • Better user experience • A modern adaptive WAF model suitable for academic or real-world use

1.4 Objective of the Project

1.4.1 General Objective

To design and implement a hybrid AI-Enhanced ModSecurity framework that integrates a Machine Learning validator to automatically classify and suppress False Positive alerts triggered by the OWASP Core Rule Set (CRS), thereby improving detection accuracy without significantly increasing request latency.

1.4.2 Specific Objectives

• To analyze the limitations of the default ModSecurity + OWASP CRS deployment by quantifying False Positive Rates (FPR) on standard datasets (e.g., CSIC 2010 or ECML/PKDD)

• To design a feature extraction pipeline that utilizes ModSecurity audit logs to extract critical features, such as triggered Rule IDs, anomaly scores, and payload structural patterns (e.g., distinguishing between JSON, SQL, and Plain Text)

• To develop and train a lightweight Machine Learning model (e.g., XGBoost, SVM, or Logistic Regression) capable of distinguishing between legitimate traffic (False Positives) and actual attacks (True Positives).

• To integrate the trained model into the ModSecurity workflow using Lua scripting to act as a "secondary opinion" mechanism that re-evaluates requests only when specific severity thresholds are met.

• To evaluate the system's effectiveness by measuring the percentage reduction in False Positives and ensuring the added processing time (latency) remains within acceptable limits for real-time web traffic.

1.5 Proposed System

The proposed solution introduces a hybrid WAF architecrue:

- Phase 1: Traditional CRS rules execute and compute anomaly score + triggered rule IDs.
- Phase 2: If anomaly score exceeds a low threshold (e.g., 3), the request features are sent to an ML inference service.
- Phase 3: ML model returns a malicious/benign prediction → final block/allow decision. This “second opinion” approach preserves the speed of rule-based detection while adding intelligence where needed.

1.6 Feasibility Study

1.6.1 Economic Feasibility

1.6.1.1 Developmental Cost

- All required tools are open-source (ModSecurity, Python, Scikit-learn).
- Costs limited to computing resources and time investment.

1.6.1.2 Operational Cost

- Low operational cost due to automation.
- Minimal maintenance needed after deployment.

1.6.2 Technical Feasibility

- Project uses accessible technologies widely supported online.
- Tools: ModSecurity v3, OWASP CRS, Python ML libraries, Flask/FastAPI.
- Logs are uncomplicated to parse and process in ML pipelines.

Thus, the project is technically achievable.

1.6.3 Schedule Feasibility

A full academic semester provides adequate time for:

- Data collection
- Model training
- Integration design
- Evaluation and documentation

1.7 Scope

In-Scope

- ModSecurity rule-based detection
- Collection and analysis of audit logs
- Machine learning classifier for malicious/benign classification.
- Real time integration.

- Evaluation metrics (precision, recall, FP reduction).

Out-of-Scope

- Full replacement of ModSecurity.
- Advanced deep learning models requiring large datasets.
- Production-grade deployment for enterprise systems

1.8 Methodology

The project follows an iterative Scrum methodology, which supports continuous refinement, frequent feedback, and incremental development. The work will be organized into repeating cycles (sprints), each focusing on a specific stage of the project. The methodology is divided into four major phases: requirement analysis, design, implementation, and testing, each executed through Scrum sprints.

1. Requirements Analysis (Initial Sprints)

The aim of this phase is to clearly understand the false-positive challenge within ModSecurity and identify all system requirements.

Activities

- Literature Review
  - Analyze research on ModSecurity, the OWASP Core Rule Set (CRS), false positives, and ML-enhanced WAFs.
  - Review existing hybrid WAF designs to adopt best practices.
- Baseline Understanding
  - Observe CRS behavior using default configurations in a controlled setup.
  - Identify which alert fields and rule behaviors are most relevant for ML classification.
- Dataset Definition
  - Obtain publicly available ModSecurity or WAF-related datasets.

Outputs

- Functional and non-functional requirements
- Defined dataset source
- Clear description of the false-positive suppression goal

2. System Design (Design Sprints)

This phase defines the architecture and high-level design choices of the hybrid WAF system.

Design Focus Areas

- Architecture Planning

  Defines the pipeline:

  - CRS rule evaluation → Feature extraction → ML validator → Decision logic.

- Constraint Identification

  - Ensure system remains efficient, compatible with ModSecurity, and maintainable over time.
  - Performance constraints emphasize minimal impact on request processing.

- Design Alternatives Review

  - Compare possible ML models and feature extraction approaches.
  - Select an architecture that balances accuracy with speed and simplicity.

- Workflow Specification

  - Define how alerts move through the system.
  - Specify how the ML "second opinion" interacts with the default CRS decision.

Outputs

- High-level architecture diagram
- Component descriptions

- Defined system workflow
- Documented design rationale

3. Implementation Methodology (Development Sprints)

This phase focuses on transforming the design into a functional system through incremental Scrum iterations.

Tools & Technologies

- WAF layer: ModSecurity with the OWASP CRS
- Model development: Python, scikit-learn or similar ML frameworks
- Integration: A small API service or connector script to link ModSecurity and the ML model
- Data processing: Python for feature extraction and preprocessing

Implementation Steps

- Environment Setup
  - Configure ModSecurity with CRS and enable audit logging.
- Dataset Preparation
  - Preprocess the downloaded datasets and extract relevant features.
- Feature Engineering
  - Define and compute the fields needed for ML classification.
- Model Training and Evaluation
  - Train multiple ML models and select the one that performs best.
- System Integration
  - Connect ModSecurity to the ML inference module to enable hybrid detection.
- Decision Logic Development

- Implement logic that uses ML predictions to assist or override CRS decisions.

Outputs

- Trained ML model
- Functional hybrid WAF prototype
- Documentation of implementation progress

4. Testing and Evaluation (Final Sprints)

This phase ensures that the system functions correctly, integrates smoothly, and meets the project goals.

Testing Types

- Functional Testing
  - Verify the logic of alert classification and decision-making.
- System Testing
  - Ensure proper communication between ModSecurity, the ML validator, and the decision engine.
- Performance Consideration
  - Confirm that the ML inference process does not cause noticeable delays.
- Model Evaluation
  - Assess model performance using standard ML metrics such as precision and recall.
- Expert Review
  - Present results to instructors or peers for feedback on clarity and architecture.

Scrum Workflow Summary

- Backlog Creation: Requirements, features, and tasks added as backlog items
- Sprint Planning: Select tasks for each sprint

• Development: Iterative progress through analysis, design, implementation, and testing

• Sprint Review: Demonstrate progress and receive feedback

• Sprint Retrospective: Improve process for the next cycle

1.9 Project Management Plan

1.9.1 Time Management Plan (Gantt Overview)

Task	Dec	Jan	Feb	Mar	Apr	May	Jun
Literature & Requirements	■	■					
Data Collection & Labeling		■	■				
Model Training & Selection			■	■			
Integration Design				■	■		
Implementation					■	■	
Testing & Evaluation						■	■
Documentation & Defense							■

1.9.2 Quality Management Plan

Risks affecting quality:

• Incorrect log parsing → bad model performance

- Unbalanced dataset → biased classification
- Misinterpretation of attack patterns

Quality assurance strategies:

- Frequent model validation
- Manual labeling of logs
- Peer review of preprocessing scripts
- Systematic testing using controlled attacks

1.9.3 Communication Management Plan

Type	Method / Tool	Frequency	Information	Participants
Internal Meetings	In-person / Telegram	Weekly	Progress, issues	Project team
Advisor Meetings	In-person/Email	Bi-weekly	Status updates	Student & Advisor
Documentation Sharing	Google Drive	Continuous	Source files & logs	Team
Final Presentation	Classroom	End of semester	Results	Instructor, students

REFERENCE

1. https://coreruleset.org/

2. https://doi.org/10.1109/SP.2010.40

3. https://coreruleset.org/blog/

4. https://blog.cloudflare.com/waf-machine-learning/